

CHENNAI INSTITUTE OF TECHNOLOGY

Sarathy Nagar, Kundrathur, Chennai-600069

An Autonomous Institute Approved by AICTE and Affiliated to Anna University, Chennai

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

CS4501 – COMPILER DESIGN



Department of Computer Science and Engineering

By
Name:
Reg No:

JULY TO DECEMBER - 2026

CHENNAI INSTITUTE OF TECHNOLOGY CHENNAI-69

Vision of the Institute:

To be an eminent centre for Academia, Industry and Research by imparting knowledge, relevant practices and inculcating human values to address global challenges through novelty and sustainability.

Mission of the Institute:

- IM1.** To create next generation leaders by effective teaching learning methodologies and instill scientific spark in them to meet the global challenges.
- IM2.** To transform lives through deployment of emerging technology, novelty and sustainability.
- IM3.** To inculcate human values and ethical principles to cater the societal needs.
- IM4.** To contribute towards the research ecosystem by providing a suitable, effective platform for interaction between industry, academia and R & D establishments.



**CHENNAI
INSTITUTE OF TECHNOLOGY**
(Autonomous)

Department of Computer Science and Engineering

Vision of the Department:

To Excel in the emerging areas of Computer Science and Engineering by imparting knowledge, relevant practices and inculcating human values to transform the students as potential resources to contribute innovatively through advanced computing in real time situations.

Mission of the Department:

DM1: To provide strong fundamentals and technical skills for Computer Science applications through effective teaching learning methodologies.

DM2: To transform lives of the students by nurturing ethical values, creativity and novelty to become Entrepreneurs and establish start-ups.

DM3: To habituate the students to focus on sustainable solutions to improve the quality of life and the welfare of the society.

DM4: To enhance the fabric of research in computing through collaborative linkages with industry and academia.

DM5: To inculcate learning of the emerging technologies to pursue higher studies leading to lifelong learning.

Register No:

--	--	--	--	--	--	--

BONAFIDE CERTIFICATE

Certified that this is the bonafide record of work done by
Mr./Ms..... of
..... semester B.E
.....Engineering
in theLaboratory during the
academic year

Staff-In-Charge

Head of the Department

Submitted for the University Practical Examination held on.....

Internal Examiner

External Examiner

Date:

Date:

INDEX

S.No.	Date	Title of the Experiment	Page No.	CO's Mapped	PO's, PSO's Mapped	Signature
1		Using the LEX tool, develop a lexical analyzer to recognize a few patterns in C. (Ex. identifiers, constants, comments, operators etc.). Create a symbol table, while recognizing identifiers.		CO1	PO1, PO2, PSO1	
2		Implement a Lexical Analyzer using LEX Tool		CO1	PO1, PO2, PSO1	
3		Program to recognize a valid arithmetic expression that uses operator +, -, * and /.		CO2	PO1, PO3, PSO1	
4		Program to recognize a valid variable which starts with a letter followed by any number of letters or digits.		CO2	PO1, PO3, PSO1	
5		Program to recognize a valid control structures syntax of C language (For loop, while loop, if-else, if-else-if, switch-case, etc.).		CO2	PO1, PO3, PSO1	
6		Implementation of calculator using LEX and YACC		CO3	PO1, PO2, PO5, PSO2	
7		Generate three address code for a simple program using LEX and YACC.		CO3	PO1, PO2, PO5, PSO2	
8		Implement type checking using Lex and Yacc.		CO4	PO1, PO2, PO5, PSO2	
9		Implement simple code optimization techniques (Constant folding, Strength reduction and Algebraic transformation)		CO4	PO1, PO2, PO3, PSO2	
10		Implement back-end of the compiler for which the three-address code is given as input and the 8086-assembly language code is produced as output.		CO5	PO1, PO2, PO3, PSO2	

EX.NO: 1

Using the LEX tool, develop a lexical analyzer to recognize a few patterns in C. (Ex. identifiers, constants, comments, operators etc.). Create a symbol table, while recognizing identifiers.

DATE:

AIM:

To develop a lexical analyzer using FLEX to recognize tokens such as identifiers, constants, comments, and operators in a C program and to create a symbol table while recognizing identifiers.

ALGORITHM:

Start the program by including the necessary headers within the FLEX definitions section (%{ ... %}).

Define regular expressions for:

- o Identifiers: [a-zA-Z_][a-zA-Z0-9_]*
- o Constants: [0-9]+
- o Comments: //.* and /* ... */
- o Operators: + - * / = < >

Declare a symbol table array (structure with name and type fields) in the definitions section.

Write rules in the rules section of the FLEX (.l) file:

- o When an identifier is recognized, call insert() to add it to the symbol table if not already present.
- o Print or categorize constants, operators, and comments as they are matched.

Use yylex() actions to print matched tokens and perform symbol table insertion.

In main(), open the input file, call yylex(), then print the symbol table.

Compile the FLEX file using flex and gcc.

Execute the program with a sample C code input file.

Stop.

PROCEDURE:

Open a terminal and create a new FLEX file, symtab.l.

In the definitions section, include headers and declare the symbol table array along with insert()/lookup() helper functions.

In the rules section, define patterns for identifiers, constants, comments, and operators.

Use { printf(...) } actions to print the recognized tokens and call insert() for identifiers.

Save and compile the file using:

```
flex symtab.l
```

```
gcc lex.yy.c -o symtab -lfl
```

Run the executable:

```
./symtab input.c
```

Observe the output and verify the tokens and symbol table entries.

PROGRAM:

```
symtab.l
%{
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

struct symtab {
    char name[30];
    int type;
} symtab[100];
int sc = 0;

int lookup(char *s) {
    int i;
    for (i = 0; i < sc; i++)
        if (strcmp(symtab[i].name, s) == 0)
            return i;
    return -1;
}
```

```

}

void insert(char *s) {
    if (lookup(s) == -1) {
        strcpy(symtab[sc].name, s);
        symtab[sc].type = 1;
        sc++;
    }
}

}%

DIGIT    [0-9]
ID       [a-zA-Z_][a-zA-Z0-9_]*

%%
"/"([^*]|\\*+[^*/])\\*+/"      { printf("Comment      : %s\n", yytext); }
"/"\\.\\*+                      { printf("Comment      : %s\n", yytext); }
{ID}                            { insert(yytext); printf("Identifier : %s\n", yytext); }
{DIGIT}+                        { printf("Constant   : %s\n", yytext); }
"+"|"-"|"*"|"/"|"="|"<"|">"  { printf("Operator   : %s\n", yytext); }
[ \\t\\n]                       { /* skip whitespace */ }
.                                { /* ignore other characters */ }
%%

int yywrap() { return 1; }

int main(int argc, char *argv[]) {
    if (argc < 2) {
        printf("Usage: %s <input file>\n", argv[0]);
        return 1;
    }
    yyin = fopen(argv[1], "r");
    if (!yyin) {
        printf("Cannot open file %s\n", argv[1]);
        return 1;
    }
    yylex();

    printf("\nSYMBOL TABLE\n");
    printf("S.No\tName\n");
    int i;
    for (i = 0; i < sc; i++)
        printf("%d\t%s\n", i + 1, symtab[i].name);

    fclose(yyin);
    return 0;
}

```

OUTPUT:

```

$ flex symtab.l
$ gcc lex.yy.c -o symtab -lfl
$ cat input.c
int a = 10; // sum variable
b = a + 5;
$ ./symtab input.c
Comment      : // sum variable
Identifier   : int
Identifier   : a
Constant     : 10
Operator     : =
Identifier   : b
Identifier   : a
Operator     : +
Constant     : 5

```

SYMBOL TABLE

S.No	Name
1	int
2	a
3	b

RESULT:

Thus the FLEX program to develop a lexical analyzer recognizing identifiers, constants, comments and operators, and to build a symbol table, was executed and verified successfully.

AIM:

The goal is to create a program that reads a C source code file and identifies individual tokens such as identifiers, keywords, constants, operators, preprocessor directives, header files and delimiters, using FLEX and its built-in regular expression matching.

ALGORITHM:

Start by defining patterns (using regular expressions) for each type of token (keywords, identifiers, numbers, operators, delimiters, etc.).

Set up a FLEX (.l) file with three parts:

- o Definitions
- o Rules
- o C code (main function)

In the rules section, match each pattern with an action (e.g., print "Keyword" if a keyword is found).

Compile the FLEX program using the flex and gcc commands.

Run the compiled program and give it some C code as input.

The program will scan the input and print out each token type.

Stop after all tokens are processed.

PROCEDURE:

Create a file with .l extension, lexer.l, where you write the FLEX code.

Inside the file, define how to recognize tokens using regular expressions.

Use the flex command to convert your .l file into a C program (lex.yy.c).

Compile the generated C program using gcc to produce an executable.

Run the executable and give it input C code.

The output will show you which tokens were recognized (Keyword, Identifier, Number, Operator, Delimiter, etc.).

PROGRAM:

```
lexer.l
%{
#include <stdio.h>
%}

KEYWORD int|float|char|double|void|for|while|if|else|return|struct|switch|case|break|do

%%
"#include"           { printf("Preprocessor Directive : %s\n", yytext); }
"<\"[a-zA-Z.]+>\""      { printf("Header File           : %s\n", yytext); }
{KEYWORD}            { printf("Keyword           : %s\n", yytext); }
[a-zA-Z_][a-zA-Z0-9_]* { printf("Identifier          : %s\n", yytext); }
[0-9]+               { printf("Number            : %s\n", yytext); }
"=="|"<="|">="        { printf("Operator           : %s\n", yytext); }
"+"|"-"|"*"|"/"|"="|"<|">" { printf("Operator           : %s\n", yytext); }
[(){};],            { printf("Delimiter          : %s\n", yytext); }
[ \t\n]              { /* skip whitespace */ }
.                    { /* ignore rest */ }
%%

int yywrap() { return 1; }

int main(int argc, char *argv[]) {
    if (argc < 2) {
        printf("Usage: %s <input file>\n", argv[0]);
        return 1;
    }
    yyin = fopen(argv[1], "r");
    yylex();
    printf("\nEnd of file\n");
    return 0;
}
```

OUTPUT:

```
$ flex lexer.l
$ gcc lex.yy.c -o lexer -lfl
$ cat iplex.c
#include<stdio.h>
void main()
{
    int x;
    x = 10;
}
$ ./lexer iplex.c
Preprocessor Directive : #include
Header File           : <stdio.h>
Keyword                : void
Identifier             : main
Delimiter              : (
Delimiter              : )
Delimiter              : {
Keyword                : int
Identifier             : x
Delimiter              : ;
Identifier             : x
Operator               : =
Number                 : 10
Delimiter              : ;
Delimiter              : }
```

End of file

RESULT:

Thus, the FLEX program for implementation of a Lexical Analyzer was executed and verified successfully.

AIM:

To write a program to recognize a valid arithmetic expression that uses operator +, -, * and / using FLEX and BISON.

ALGORITHM:

FLEX

1. Declare the required header file and variable declaration within '%{' and '%}'.
2. FLEX requires regular expressions to identify valid arithmetic expression token of lexemes.
3. FLEX calls yywrap() function after input is over. It should return 1 when work is done or should return 0 when more processing is required.

BISON

1. Declare the required header file and variable declaration within '%{' and '%}'.
2. Define tokens in the first section and also define the associativity of the operations.
3. Mention the grammar productions and the action for each production.
4. \$\$ refers to the top of the stack position while \$1, \$2 refer to respective values in the stack.
5. Call yyparse() to initiate the parsing process.
6. yyerror() function is called when no productions in the grammar match the input statement.

PROCEDURE:

Open a text editor and write the FLEX source file art_expr.l.

Define regular expressions for identifiers, digits, operators, and ignore whitespaces.

Save and close the FLEX file.

Open another text file and write the BISON source file art_expr.y.

Define tokens and grammar rules to parse arithmetic expressions using +, -, *, and /.

Save and close the BISON file.

Use the following commands to compile and run:

- o flex art_expr.l
- o bison -d art_expr.y
- o gcc lex.yy.c art_expr.tab.c -o art_expr -lfl
- o ./art_expr

Enter expressions as input. If the expression is valid, it displays "valid Expression", otherwise "Invalid Expression".

PROGRAM:

```
//art_expr.l
%{
#include<stdio.h>
#include "art_expr.tab.h"
%}
%%
[a-zA-Z][0-9a-zA-Z]* {return ID;}
[0-9]+ {return DIG;}
[ \t]+ {;}
. {return yytext[0];}
\n {return 0;}
%%
int yywrap() {
return 1;
}

//art_expr.y
%{
#include<stdio.h>
%}
%token ID DIG
%left '+' '-'
%left '*' '/'
```

```

%right UMINUS
%%
stmt: expn ;
expn: expn '+' expn | expn '-' expn | expn '*' expn | expn '/' expn | '-' expn %prec UMINUS |
      '(' expn ')' | DIG | ID;
%%
int main() {
printf("Enter the Expression\n");
yyparse();
printf("valid Expression\n");
return 0;
}
int yyerror() {
printf("Invalid Expression");
exit(0);
}

```

OUTPUT:

```

"art_expr.y" 33L, 372C written
[gomathy@rhel5 ~]$ flex art_expr.l
[gomathy@rhel5 ~]$ bison -d art_expr.y
[gomathy@rhel5 ~]$ gcc lex.yy.c art_expr.tab.c -o art_expr -lfl
[gomathy@rhel5 ~]$ ./art_expr
Enter the Expression
a+b*c-d/e
valid Expression
[gomathy@rhel5 ~]$ ./art_expr
Enter the Expression
a=b
Invalid Expression
[gomathy@rhel5 ~]$ _

```

RESULT:

Thus the program to recognize a valid arithmetic expression that uses operator +, -, *, and / using FLEX and BISON was executed and verified successfully.

AIM:

To write a program to recognize a valid variable which starts with a letter followed by any number of letters or digits using FLEX and BISON.

ALGORITHM:**FLEX**

1. Declare the required header file and variable declaration with in '%{' and '%}'.
2. FLEX requires regular expressions or patterns to identify token of lexemes for recognizing a valid variable.
3. FLEX calls yywrap() function after input is over. It should return 1 when work is done or should return 0 when more processing is required.

BISON

1. Declare the required header file and variable declaration within '%{' and '%}'.
2. Define tokens in the first section and also define the associativity of the operations.
3. Mention the grammar productions and the action for each production.
4. \$\$ refers to the top of the stack position while \$1 refers to the first value, \$2 for the second value in the stack.
5. Call yyparse() to initiate the parsing process.
6. yyerror() function is called when none of the productions in the grammar match the input statement.

PROCEDURE:

Create the FLEX file valvar.l using a text editor.

Define token patterns for letters and digits, and return them as LET and DIG tokens.

Save and close the file.

Create the BISON file valvar.y.

Define grammar rules that recognize valid variable names (starting with a letter and followed by letters or digits).

Save and close the BISON file.

Open terminal/command prompt and compile using:

- o flex valvar.l
- o bison -d valvar.y
- o gcc lex.yy.c valvar.tab.c -o valvar -lfl

Run the program: ./valvar

Enter test inputs like abc1, var123, labc to test validation.

Observe whether the variable is valid or invalid.

PROGRAM:

```
//valvar.l
%{
#include "valvar.tab.h"
%}
%%
[a-zA-Z] {return LET;}
[0-9] {return DIG;}
\n {return 0;}
%%
int yywrap() {
return 1;
}

//valvar.y
%{
#include<stdio.h>
%}
%token LET DIG
%%
variable: var ;
var: var DIG | var LET | LET ;
```

```

%%
int main() {
printf("Enter the variable:\n");
yyparse();
printf("Valid variable\n");
return 0;
}
int yyerror() {
printf("Invalid variable\n");
exit(0);
}

```

OUTPUT:

```

"valvar.y" 25L, 246C written
[gomathy@rhel5 ~]$ flex valvar.l
[gomathy@rhel5 ~]$ bison -d valvar.y
[gomathy@rhel5 ~]$ gcc lex.yy.c valvar.tab.c -o valvar -lfl
[gomathy@rhel5 ~]$ ./valvar
Enter the variable:
add
Valid variable
[gomathy@rhel5 ~]$ ./valvar
Enter the variable:
add1
Valid variable
[gomathy@rhel5 ~]$ ./valvar
Enter the variable:
1add
Invalid variable
[gomathy@rhel5 ~]$

```

RESULT:

Thus the program to recognize a valid variable which starts with a letter followed by any number of letters or digits using FLEX and BISON was executed and verified successfully.

AIM:

To write a program to recognize a valid control structure syntax of C language (such as for loop, while loop, if-else, if-else-if, switch-case, etc.) using FLEX and BISON.

ALGORITHM:

FLEX:

1. Start the FLEX program with required header files and token declarations.
2. Define regular expressions for control keywords such as if, else, for, while, switch, case, etc.
3. Return appropriate tokens for each keyword to BISON.

BISON:

1. Include header files and token declarations.
2. Define grammar rules to match valid syntax for:
 - o if statement
 - o if-else and if-else-if ladder
 - o while and for loops
 - o switch-case structure
3. Implement yyparse() to start parsing and yyerror() for invalid inputs.

PROCEDURE:

Write the control.l file using FLEX to tokenize control structure keywords.

Write the control.y file using BISON to define grammar rules for C control structures.

Compile and execute the program using:

- o flex control.l
- o bison -d control.y
- o gcc lex.yy.c control.tab.c -o control -lfl
- o ./control

Input sample control structures in C-style syntax to check for validation.

PROGRAM:

```
control.l
%{
#include "control.tab.h"
%}
%%
"if" { return IF; }
"else" { return ELSE; }
"for" { return FOR; }
"while" { return WHILE; }
"switch" { return SWITCH; }
"case" { return CASE; }
"default" { return DEFAULT; }
[a-zA-Z_][a-zA-Z0-9_]* { return ID; }
[0-9]+ { return NUM; }
"{" { return LBRACE; }
"}" { return RBRACE; }
"(" { return LPAREN; }
")" { return RPAREN; }
":" { return COLON; }
";" { return SEMICOLON; }
"==" { return EQ; }
"<=" { return LE; }
">=" { return GE; }
"<" { return LT; }
">" { return GT; }
"=" { return ASSIGN; }
[ \t\n] { /* skip whitespace */ }
. { return yytext[0]; }
```

```

%%
int yywrap() { return 1; }

control.y
%{
#include <stdio.h>
#include <stdlib.h>
%}
%token IF ELSE FOR WHILE SWITCH CASE DEFAULT
%token ID NUM
%token LBRACE RBRACE LPAREN RPAREN COLON SEMICOLON
%token EQ LE GE LT GT ASSIGN
%%
program:
stmt_list
;
stmt_list:
stmt_list stmt
| stmt
;
stmt:
if_stmt
| while_stmt
| for_stmt
| switch_stmt
;
if_stmt:
IF LPAREN cond RPAREN stmt
| IF LPAREN cond RPAREN stmt ELSE stmt
;
while_stmt:
WHILE LPAREN cond RPAREN stmt
;
for_stmt:
FOR LPAREN ID ASSIGN NUM SEMICOLON cond SEMICOLON ID ASSIGN ID RPAREN stmt
;
switch_stmt:
SWITCH LPAREN ID RPAREN LBRACE case_list RBRACE
;
case_list:
case_list CASE NUM COLON stmt
| case_list DEFAULT COLON stmt
| CASE NUM COLON stmt
| DEFAULT COLON stmt
;
cond:
ID relop NUM
;
relop:
EQ | LE | GE | LT | GT
;
%%
int main() {
printf("Enter a C control structure syntax:\n");
yyvsparse();
printf("Valid control structure syntax.\n");
return 0;
}
int yyerror(char *s) {
printf("Invalid control structure syntax.\n");
return 0;
}

```

OUTPUT:

```

$ flex control.l
$ bison -d control.y
$ gcc lex.yy.c control.tab.c -o control -lfl
$ ./control
Enter a C control structure syntax:
if (x < 5) { y = 10; }
Valid control structure syntax.

```


RESULT:

Thus the program to recognize a valid control structure syntax of C language (For loop, while loop, if-else, if-else-if, switch-case, etc.) using FLEX and BISON was executed and verified successfully.

AIM:

To write a program to implement a Calculator using FLEX and BISON.

ALGORITHM:

Step 1: Start the program.

Step 2: In the definitions part of the FLEX file, include the regular definition for a digit.

Step 3: In the rules part of the FLEX file, specify the pattern for a number and its action (return NUM with the numeric value in yylval) in cal.l.

Step 4: In the BISON program, define grammar rules so that all the arithmetic operations +, -, *, / are evaluated using operator precedence.

Step 5: Display error if the input does not match the grammar.

Step 6: Provide the input.

Step 7: Verify the output.

Step 8: End.

PROCEDURE:

Create a file named cal.l and define patterns to identify numbers using regular expressions.

For matched digits, return the token NUM and store the number using yylval.

Create another file named cal.y to define grammar rules for arithmetic expressions using BISON.

Include operator precedence and associativity using %left and %right.

Add rules to evaluate expressions like $E + E$, $E - E$, $E * E$, and E / E .

Compile both files using the following commands:

- o flex cal.l
- o bison -d cal.y
- o gcc lex.yy.c cal.tab.c -o calc -lfl

Run the compiled program: ./calc

Enter arithmetic expressions and view the result.

Test with multiple expressions to verify both valid and invalid cases.

PROGRAM:

```
cal.l
%{
#include "cal.tab.h"
%}
DIGIT [0-9]+
%option noyywrap
%%
{DIGIT} { yylval = atof(yytext); return NUM; }
\n|. { return yytext[0]; }
%%
```

```
cal.y
%{
#include <ctype.h>
#include <stdio.h>
#define YYSTYPE double
%}
%token NUM
%left '+' '-'
%left '*' '/'
%right UMINUS
%%
Statement: E { printf("Answer: %g \n", $$); }
| Statement '\n'
;
E : E '+' E { $$ = $1 + $3; }
| E '-' E { $$ = $1 - $3; }
| E '*' E { $$ = $1 * $3; }
```

```
| E '/' E { $$ = $1 / $3; }
| NUM
;
%%
int main() {
printf("Enter the expression:\n");
yyparse();
return 0;
}
int yyerror(char *s) {
printf("%s\n", s);
return 0;
}
```

OUTPUT:

```
$ flex cal.l
$ bison -d cal.y
$ gcc lex.yy.c cal.tab.c -o calc -lfl
$ ./calc
Enter the expression:
2+2
Answer: 4
```

RESULT:

Thus the program for implementing a calculator using FLEX and BISON was executed and verified successfully.

AIM:

To write a program using FLEX and BISON to generate three-address code (TAC) for a simple arithmetic expression.

ALGORITHM:

FLEX:

1. Include required headers and define tokens for identifiers, numbers, and operators.
2. Use regular expressions to identify identifiers and numeric constants.
3. Return appropriate tokens to BISON for parsing.

BISON:

1. Declare tokens and define associativity for operators.
2. Use grammar rules to parse arithmetic expressions (e.g., $a = b + c * d$).
3. Generate three-address code during the parsing actions.
4. Maintain a temporary variable counter to represent intermediate results (e.g., $t1 = b * d$).

PROCEDURE:

Create the FLEX file tac.l:

- o Tokenize input using patterns for identifiers, numbers, and operators.
- o Pass tokens to BISON.

Create the BISON file tac.y:

- o Parse arithmetic expressions.
- o Generate three-address code using temporary variables (t1, t2, etc.) during parsing.

Compile and run:

```
flex tac.l
```

```
bison -d tac.y
```

```
gcc tac.tab.c lex.yy.c -o tac -lfl
```

```
./tac
```

Input an arithmetic expression like:

```
a = b + c * d
```

View generated three-address code.

PROGRAM:

```
tac.l
%{
#include "tac.tab.h"
#include <string.h>
%}
%%
[a-zA-Z][a-zA-Z0-9]* { yylval.str = strdup(yytext); return ID; }
[0-9]+ { yylval.str = strdup(yytext); return NUM; }
[\\t\\n ]+ { /* skip spaces */ }
. { return yytext[0]; }
%%

int yywrap() {
return 1;
}
```

```
tac.y
%{
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
int tempCount = 1;
char temp[10];
%}
%union { char *str; }
%token <str> ID NUM
```

```

%type <str> expr
%left '+' '-'
%left '*' '/'
%%
stmt: ID '=' expr { printf("%s = %s\n", $1, $3); }
;
expr: expr '+' expr {
    sprintf(temp, "t%d", tempCount++);
    printf("%s = %s + %s\n", temp, $1, $3);
    $$ = strdup(temp);
}
| expr '-' expr {
    sprintf(temp, "t%d", tempCount++);
    printf("%s = %s - %s\n", temp, $1, $3);
    $$ = strdup(temp);
}
| expr '*' expr {
    sprintf(temp, "t%d", tempCount++);
    printf("%s = %s * %s\n", temp, $1, $3);
    $$ = strdup(temp);
}
| expr '/' expr {
    sprintf(temp, "t%d", tempCount++);
    printf("%s = %s / %s\n", temp, $1, $3);
    $$ = strdup(temp);
}
| ID { $$ = $1; }
| NUM { $$ = $1; }
;
%%
int main() {
    printf("Enter the expression:\n");
    yyparse();
    return 0;
}
int yyerror(char* s) {
    printf("Error: %s\n", s);
    return 0;
}

```

OUTPUT:

```

$ flex tac.l
$ bison -d tac.y
$ gcc tac.tab.c lex.yy.c -o tac -lfl
$ ./tac
Enter the expression:
a = b + c * d
t1 = c * d
t2 = b + t1
a = t2

```

RESULT:

Thus, the program to generate three-address code using FLEX and BISON was executed and verified successfully.

AIM:

To write a program using FLEX and BISON to implement type checking of variables in simple declarations and expressions, using a symbol table built during parsing.

ALGORITHM:

Start the program.

Use FLEX to tokenize keywords (int, float), identifiers, numbers and operators, passing them to BISON.

In BISON, define grammar rules for declaration statements and assignment statements.

On a declaration (e.g. int a;), insert the variable name and its type into the symbol table.

On an assignment (e.g. a = b + c;), look up the types of the result variable and operands in the symbol table.

If a variable used is not found in the symbol table, report it as undefined.

If all types match, print "No type mismatch"; otherwise print "Type mismatch".

End the program.

PROCEDURE:

Create FLEX File (typecheck.l)

- o Define patterns for keywords (int, float), identifiers, numbers, and operators.
- o Return these tokens to BISON for further processing.

Create BISON File (typecheck.y)

- o Define grammar rules for declarations and expressions.
- o Maintain a symbol table (array of name/type pairs) that is filled in on each declaration.
- o On each assignment, verify that the operand types and result type match and print the outcome.

Compile the FLEX and BISON Files

```
flex typecheck.l
```

```
bison -d typecheck.y
```

```
gcc lex.yy.c typecheck.tab.c -o typecheck -lfl
```

Run the Program

```
./typecheck
```

- o Input variable declarations and expressions, ending each statement with ';'.
 - o Observe output showing the type-checking result.

Test with Various Inputs

- o Try correctly typed expressions.
- o Try mismatched types (e.g., int a; float b; a = b + 1;).
- o Try using undeclared variables.

End the Program

PROGRAM:

```
typecheck.l
%{
#include "typecheck.tab.h"
#include <string.h>
#include <stdlib.h>
%}
%%
"int"          { return INT; }
"float"        { return FLOAT; }
[a-zA-Z_][a-zA-Z0-9_]*
[0-9]+         { yylval.str = strdup(yytext); return ID; }
              { yylval.str = strdup(yytext); return NUM; }
"="           { return '='; }
"+"          { return '+'; }
"-"          { return '-'; }
"*"          { return '*'; }
"/"          { return '/'; }
";"          { return ';'; }
[ \t\n]       { /* skip whitespace */ }
```

```

%%
int yywrap() { return 1; }

typecheck.y
%{
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

struct sym { char name[20]; char type[10]; } table[50];
int n = 0;

void insert(char *name, char *type) {
    strcpy(table[n].name, name);
    strcpy(table[n].type, type);
    n++;
}

char* typeOf(char *name) {
    int i;
    for (i = 0; i < n; i++)
        if (strcmp(table[i].name, name) == 0)
            return table[i].type;
    return "undefined";
}
%}
%union { char *str; }
%token <str> ID NUM
%token INT FLOAT
%type <str> expr
%%
program: stmts ;
stmts: stmts stmt | stmt ;
stmt: decl | assign ;
decl:
    INT ID ';' { insert($2, "int"); }
  | FLOAT ID ';' { insert($2, "float"); }
  ;
assign:
    ID '=' expr ';' {
        char *lt = typeOf($1);
        if (strcmp(lt, "undefined") == 0)
            printf("Undefined variable: %s\n", $1);
        else if (strcmp(lt, $3) == 0)
            printf("No type mismatch in expression: %s = ...\n", $1);
        else
            printf("Type mismatch in assignment to %s\n", $1);
    }
  ;
expr:
    ID {
        char *t = typeOf($1);
        if (strcmp(t, "undefined") == 0)
            printf("Undefined variable: %s\n", $1);
        $$ = t;
    }
  | NUM { $$ = "int"; }
  | expr '+' expr { $$ = (strcmp($1, $3) == 0) ? $1 : "mismatch"; }
  | expr '-' expr { $$ = (strcmp($1, $3) == 0) ? $1 : "mismatch"; }
  | expr '*' expr { $$ = (strcmp($1, $3) == 0) ? $1 : "mismatch"; }
  | expr '/' expr { $$ = (strcmp($1, $3) == 0) ? $1 : "mismatch"; }
  ;
%%
int main() {
    printf("Enter declarations and expressions:\n");
    yyparse();
    return 0;
}
int yyerror(char *s) {
    printf("Syntax Error: %s\n", s);
    return 0;
}

```

OUTPUT:

```
$ flex typecheck.l
$ bison -d typecheck.y
$ gcc lex.yy.c typecheck.tab.c -o typecheck -lfl
$ ./typecheck
Enter declarations and expressions:
int a;
int b;
int c;
a = b * c;
No type mismatch in expression: a = ...

$ ./typecheck
Enter declarations and expressions:
int a;
float b;
int c;
a = b + c;
Type mismatch in assignment to a
```

RESULT:

Thus, the FLEX and BISON program for type checking was successfully implemented. The program builds a symbol table from declarations and checks type consistency in assignment expressions.

AIM:

To write a program using FLEX and BISON to implement simple code optimization techniques such as constant folding, strength reduction and algebraic simplification, applied while parsing three-address code style assignment statements.

ALGORITHM:

Use FLEX to tokenize the input statements into identifiers, numbers and operators, passing them to BISON.

In BISON, define a grammar for assignment statements of the form $id = expr$;

While reducing an expr production:

- o Constant Folding: if both operands are numeric constants, evaluate the operation immediately and replace it with the result.

- o Algebraic Simplification: apply rules such as $x + 0 \rightarrow x$, $x - 0 \rightarrow x$, $x * 1 \rightarrow x$, $x / 1 \rightarrow x$.

- o Strength Reduction: replace $x * 2$ with $x + x$.

Print the optimized right-hand side once the statement is fully reduced.

Repeat for every statement until the input ends.

PROCEDURE:

Create the FLEX file optimize.l to tokenize identifiers, numbers, and the operators $+ - * / = ;$.

Create the BISON file optimize.y with a grammar for assignment statements and expressions.

In the semantic actions for each expr rule, check whether constant folding, strength reduction, or algebraic simplification applies, and print a comment indicating which optimization fired.

Compile using:

- o flex optimize.l

- o bison -d optimize.y

- o gcc lex.yy.c optimize.tab.c -o optimize -lfl

Run ./optimize and enter three-address-code-style statements ending with ';', terminate input with Ctrl+D.

Verify that constants are folded, $x*1 / x/1 / x+0 / x-0$ are simplified, and $x*2$ is converted to $x+x$.

Test with multiple statements to confirm all three optimization types are triggered correctly.

PROGRAM:

```
optimize.l
%{
#include "optimize.tab.h"
#include <string.h>
#include <stdlib.h>
%}
%%
[a-zA-Z][a-zA-Z0-9]* { yylval.str = strdup(yytext); return ID; }
[0-9]+                { yylval.str = strdup(yytext); return NUM; }
"="                  { return '='; }
"+"                  { return '+'; }
"-"                  { return '-'; }
"*"                  { return '*'; }
"/"                  { return '/'; }
";"                  { return ';'; }
[ \t\n]              { /* skip whitespace */ }
%%
int yywrap() { return 1; }
```

```
optimize.y
%{
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <ctype.h>
%}
%union { char *str; }
```

```

%token <str> ID NUM
%type <str> expr
%left '+' '-'
%left '*' '/'
%%
stmt_list: stmt_list stmt | stmt ;
stmt: ID '=' expr ';' { printf("%s = %s\n", $1, $3); } ;
expr:
    NUM { $$ = $1; }
  | ID { $$ = $1; }
  | expr '+' expr {
      if (isdigit($1[0]) && isdigit($3[0])) {
          char buf[20];
          sprintf(buf, "%d", atoi($1) + atoi($3));
          $$ = strdup(buf);
          printf("// Constant Folding: %s+%s -> %s\n", $1, $3, $$);
      } else if (strcmp($3, "0") == 0) {
          $$ = $1;
          printf("// Algebraic Simplification: x+0 -> x\n");
      } else if (strcmp($1, "0") == 0) {
          $$ = $3;
          printf("// Algebraic Simplification: 0+x -> x\n");
      } else {
          char buf[40];
          sprintf(buf, "%s + %s", $1, $3);
          $$ = strdup(buf);
      }
  }
  | expr '-' expr {
      if (isdigit($1[0]) && isdigit($3[0])) {
          char buf[20];
          sprintf(buf, "%d", atoi($1) - atoi($3));
          $$ = strdup(buf);
          printf("// Constant Folding: %s-%s -> %s\n", $1, $3, $$);
      } else if (strcmp($3, "0") == 0) {
          $$ = $1;
          printf("// Algebraic Simplification: x-0 -> x\n");
      } else {
          char buf[40];
          sprintf(buf, "%s - %s", $1, $3);
          $$ = strdup(buf);
      }
  }
  | expr '*' expr {
      if (isdigit($1[0]) && isdigit($3[0])) {
          char buf[20];
          sprintf(buf, "%d", atoi($1) * atoi($3));
          $$ = strdup(buf);
          printf("// Constant Folding: %s*%s -> %s\n", $1, $3, $$);
      } else if (strcmp($3, "1") == 0) {
          $$ = $1;
          printf("// Algebraic Simplification: x*1 -> x\n");
      } else if (strcmp($3, "2") == 0) {
          char buf[40];
          sprintf(buf, "%s + %s", $1, $1);
          $$ = strdup(buf);
          printf("// Strength Reduction: x*2 -> x+x\n");
      } else {
          char buf[40];
          sprintf(buf, "%s * %s", $1, $3);
          $$ = strdup(buf);
      }
  }
  | expr '/' expr {
      if (isdigit($1[0]) && isdigit($3[0])) {
          char buf[20];
          sprintf(buf, "%d", atoi($1) / atoi($3));
          $$ = strdup(buf);
          printf("// Constant Folding: %s/%s -> %s\n", $1, $3, $$);
      } else if (strcmp($3, "1") == 0) {
          $$ = $1;
          printf("// Algebraic Simplification: x/1 -> x\n");
      }
  }

```

```

        } else {
            char buf[40];
            sprintf(buf, "%s / %s", $1, $3);
            $$ = strdup(buf);
        }
    }
}
;
%%
int main() {
    printf("Enter Three Address Code statements (end with Ctrl+D):\n");
    yyparse();
    return 0;
}
int yyerror(char *s) {
    printf("Syntax Error: %s\n", s);
    return 0;
}

```

SAMPLE INPUT:

```

a = 2 + 4;
b = d * 1;
c = s * 2;

```

OUTPUT:

```

$ flex optimize.l
$ bison -d optimize.y
$ gcc lex.yy.c optimize.tab.c -o optimize -lfl
$ ./optimize
Enter Three Address Code statements (end with Ctrl+D):
a = 2 + 4;
// Constant Folding: 2 + 4 -> 6
a = 6
b = d * 1;
// Algebraic Simplification: x * 1 -> x
b = d
c = s * 2;
// Strength Reduction: x * 2 -> x + x
c = s + s

```

RESULT:

Thus, the FLEX and BISON program for simple code optimization techniques - constant folding, strength reduction, and algebraic simplification - was successfully implemented and tested with various inputs.

AIM:

To write a program using FLEX and BISON to implement the back-end of a compiler which takes three-address code (TAC) as input and generates equivalent 8086 assembly language code.

ALGORITHM:

Use FLEX to tokenize each TAC line into identifiers and the operators = + - * / ; passing tokens to BISON.

In BISON, define a grammar for assignment statements: id = expr ;

While reducing expr:

- o On the first operand, emit MOV AX, operand.
- o On '+', emit ADD AX, operand; on '-', emit SUB AX, operand.
- o On '*', emit MUL operand; on '/', emit MOV DX,0 / MOV BX,operand / DIV BX.

When the full statement is reduced, emit MOV result, AX.

Repeat for every TAC statement in the input and print all generated instructions.

End.

PROCEDURE:

Create the FLEX file backend.l to tokenize identifiers and operators from TAC lines.

Create the BISON file backend.y with a grammar for TAC assignment statements, embedding 8086 MOV/ADD/SUB/MUL/DIV instruction generation directly in the semantic actions.

Compile and run:

flex backend.l

bison -d backend.y

gcc lex.yy.c backend.tab.c -o backend -lfl

./backend

Input TAC statements such as: t1 = a + b followed by x = t1

View the generated 8086 assembly code.

PROGRAM:

```
backend.l
%{
#include "backend.tab.h"
#include <string.h>
#include <stdlib.h>
%}
%%
[a-zA-Z][a-zA-Z0-9]* { yylval.str = strdup(yytext); return ID; }
"=" { return '='; }
"+" { return '+'; }
"-" { return '-'; }
"*" { return '*'; }
"/" { return '/'; }
";" { return ';'; }
[ \t\n] { /* skip whitespace */ }
%%
int yywrap() { return 1; }
```

```
backend.y
%{
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
%}
%union { char *str; }
%token <str> ID
%type <str> expr
%left '+' '-'
%left '*' '/'
```

```

%%
stmt_list: stmt_list stmt | stmt ;
stmt: ID '=' expr ';' { printf("MOV %s, AX\n\n", $1); } ;
expr:
    ID { printf("MOV AX, %s\n", $1); $$ = $1; }
  | expr '+' ID { printf("ADD AX, %s\n", $3); $$ = $3; }
  | expr '-' ID { printf("SUB AX, %s\n", $3); $$ = $3; }
  | expr '*' ID { printf("MUL %s\n", $3); $$ = $3; }
  | expr '/' ID { printf("MOV DX, 0\nMOV BX, %s\nDIV BX\n", $3); $$ = $3; }
  ;
%%
int main() {
    printf("Enter TAC statements (end with Ctrl+D):\n");
    yyparse();
    return 0;
}
int yyerror(char *s) {
    printf("Syntax Error: %s\n", s);
    return 0;
}

```

SAMPLE INPUT:

```

t1 = a + b;
t2 = t1 - c;
t3 = t2 * d;
t4 = t3 / e;
x = t4;

```

OUTPUT:

```

$ flex backend.l
$ bison -d backend.y
$ gcc lex.yy.c backend.tab.c -o backend -lfl
$ ./backend

```

Enter TAC statements (end with Ctrl+D):

```

t1 = a + b;
t2 = t1 - c;
t3 = t2 * d;
t4 = t3 / e;
x = t4;

```

```

MOV AX, a
ADD AX, b
MOV t1, AX

```

```

MOV AX, t1
SUB AX, c
MOV t2, AX

```

```

MOV AX, t2
MUL d
MOV t3, AX

```

```

MOV AX, t3
MOV DX, 0
MOV BX, e
DIV BX
MOV t4, AX

```

```

MOV AX, t4
MOV x, AX

```

RESULT:

Thus, the back-end of the compiler was successfully implemented using FLEX and BISON to translate three-address code into equivalent 8086 assembly language code.